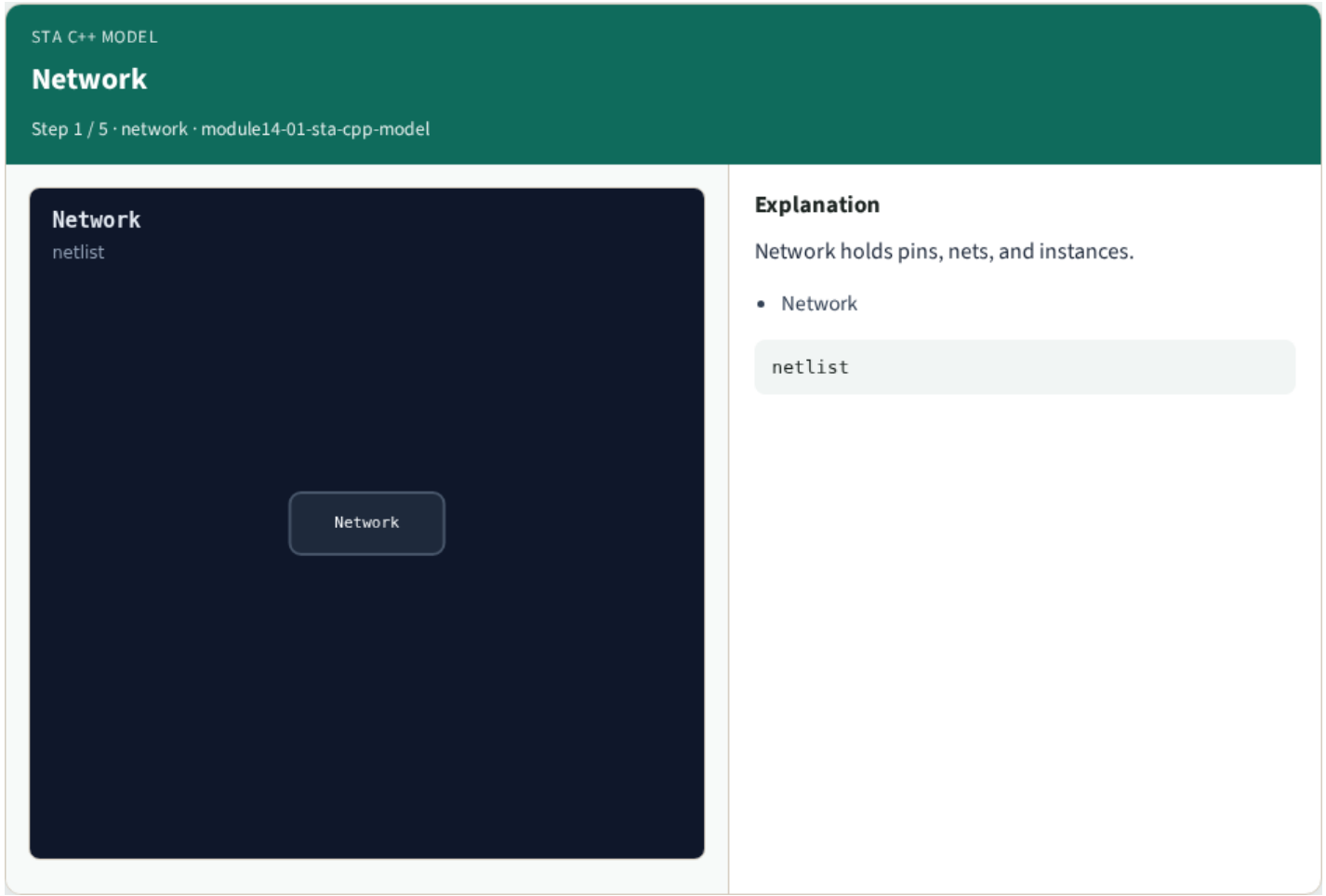
Why STA C++

Browser IR and JSON are maps

The idea

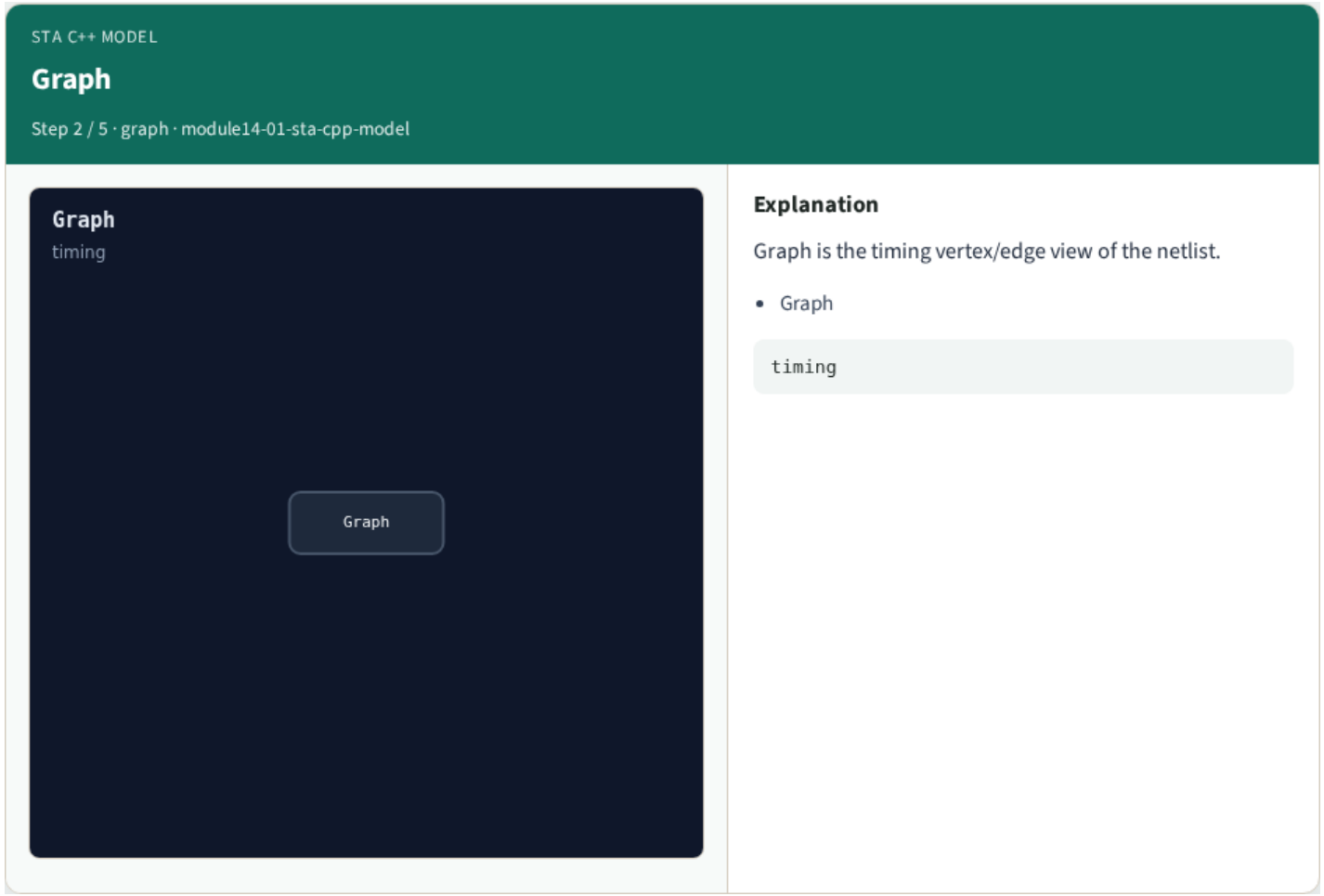
- GOLDEN: Network, Graph, Sdc, Search , Search drives arrivals and path reports

Network



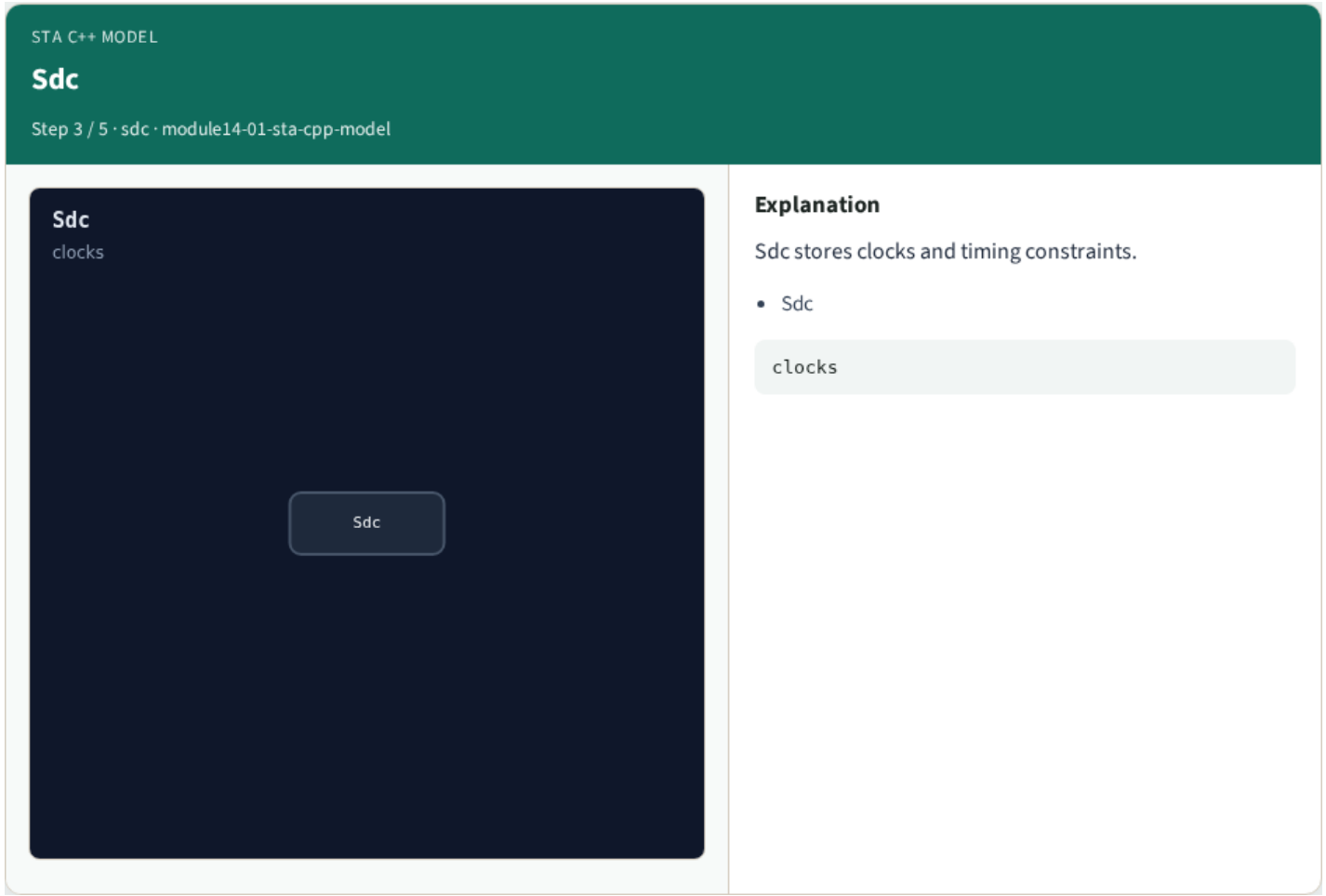
Network

Graph


STA C++ MODEL
Graph
Step 2 / 5 · graph · module14-01-sta-cpp-model
Graph
timing
Graph
Explanation
Graph is the timing vertex/edge view of the netlist.
Graph
timing

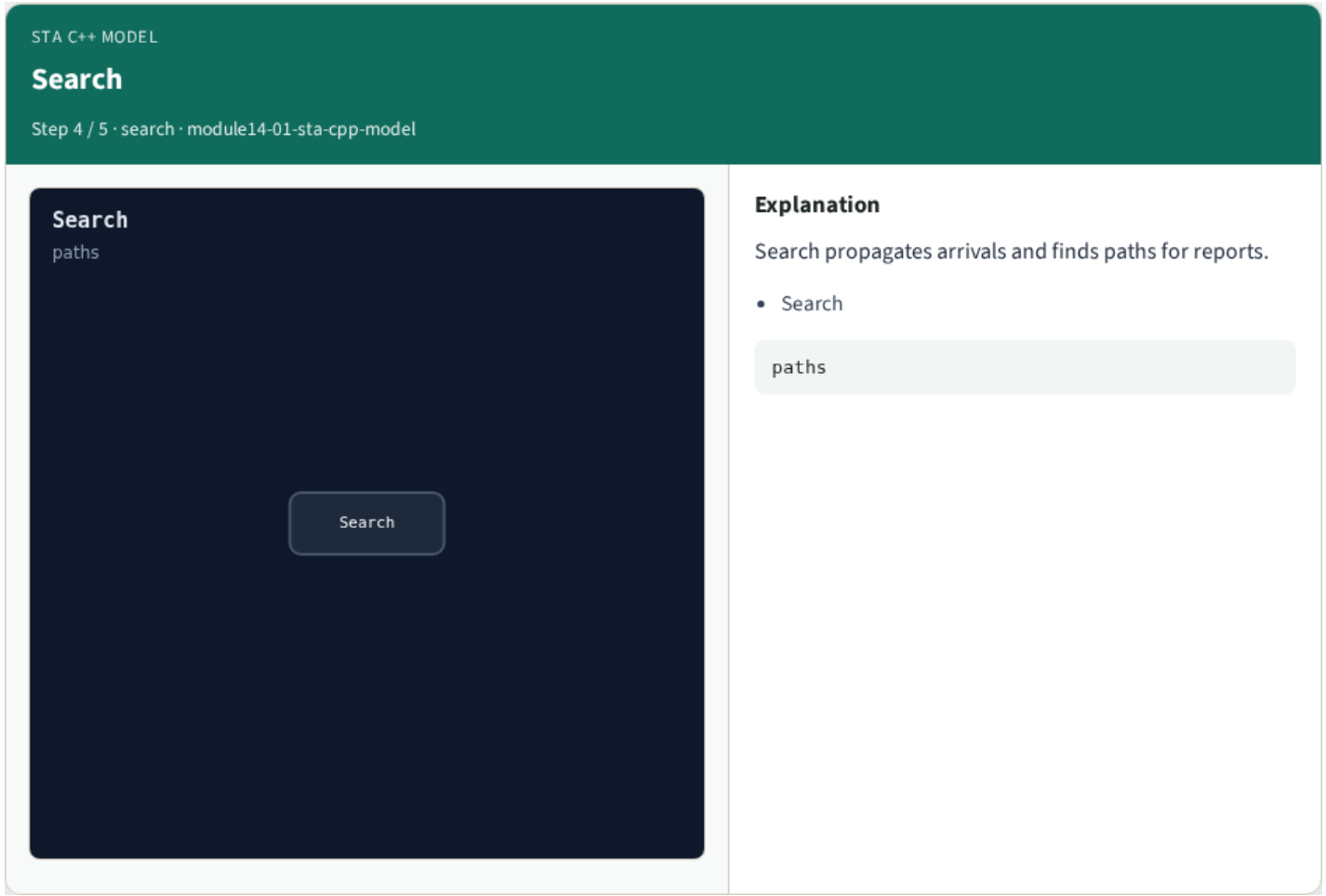
Graph

Sdc


STA C++ MODEL
Sdc
Step 3 / 5 · sdc · module14-01-sta-cpp-model
Sdc
clocks
Sdc
Explanation
Sdc stores clocks and timing constraints.
Sdc
clocks

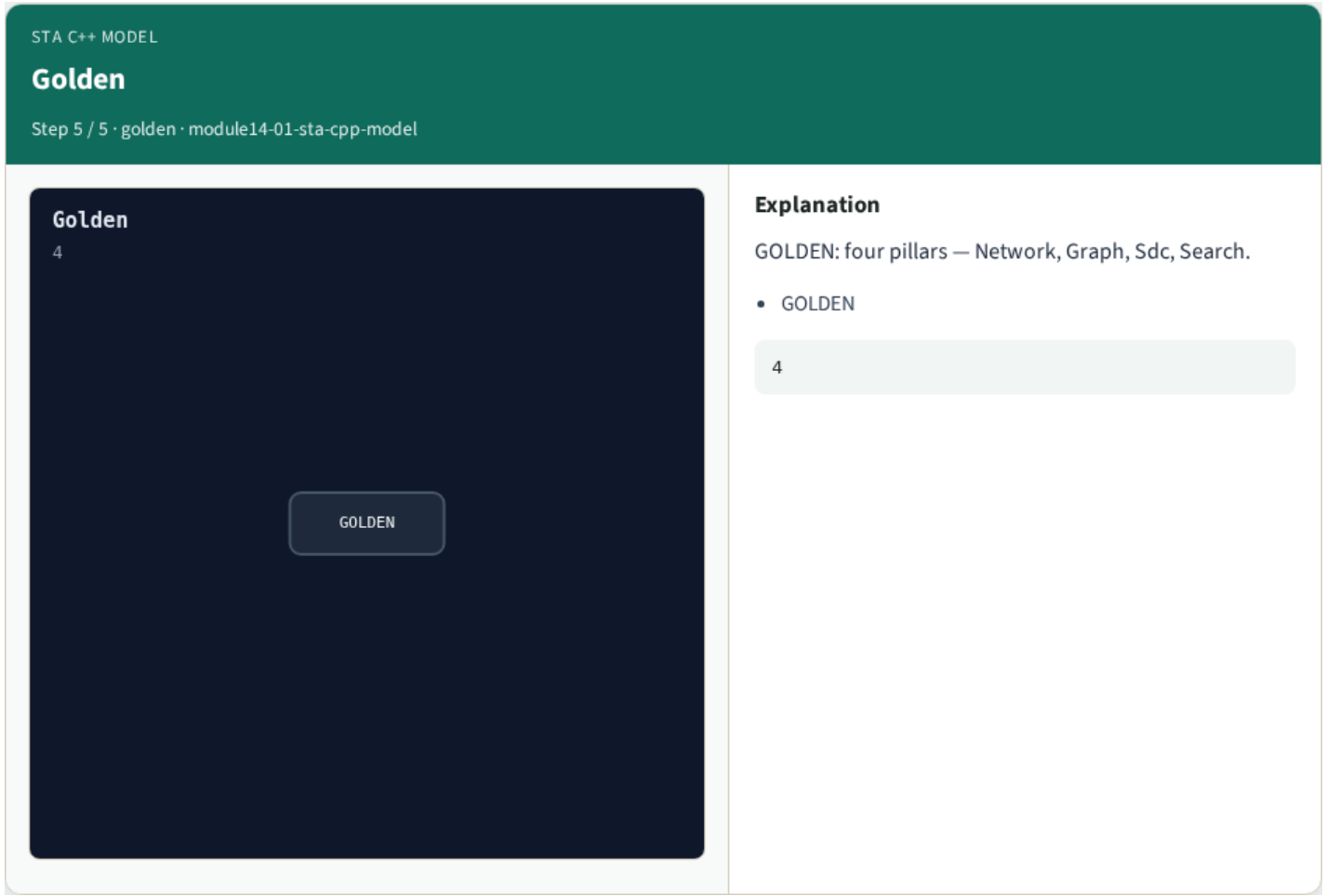
Sdc

Search



Search

Golden



Golden

Browser lab track

- Open **sta-cpp-model**
- Click through the four pillars and check the goldens

Implement track

- Call sta_cpp_model() in internals_solvers; assert pillar count and nesting

Pitfalls

- Treating JSON keys as the C++ API
- Confusing Graph with physical Magick layout

Try these

- From the common folder, run the internals solvers self-test

Try these — try these

```
cd courses/learn_opensta/common
python3 internals_solvers.py
```

Pseudocode

- Here is the OpenSTA nesting in plain steps

Pseudocode — try these

```
Network → pins/nets/instances
Graph → vertices/edges from Network
Sdc → clocks/constraints
Search → arrivals + FindPath for reports
```

Next

- Complete the quiz for this part
- Next: read the FindPath to report_checks shape in the OpenSTA tree